

Customer Churn Prediction: XGBoost vs. MLP

CSC3173 — Artificial Intelligence (2024/25) — Programming Assignment 01

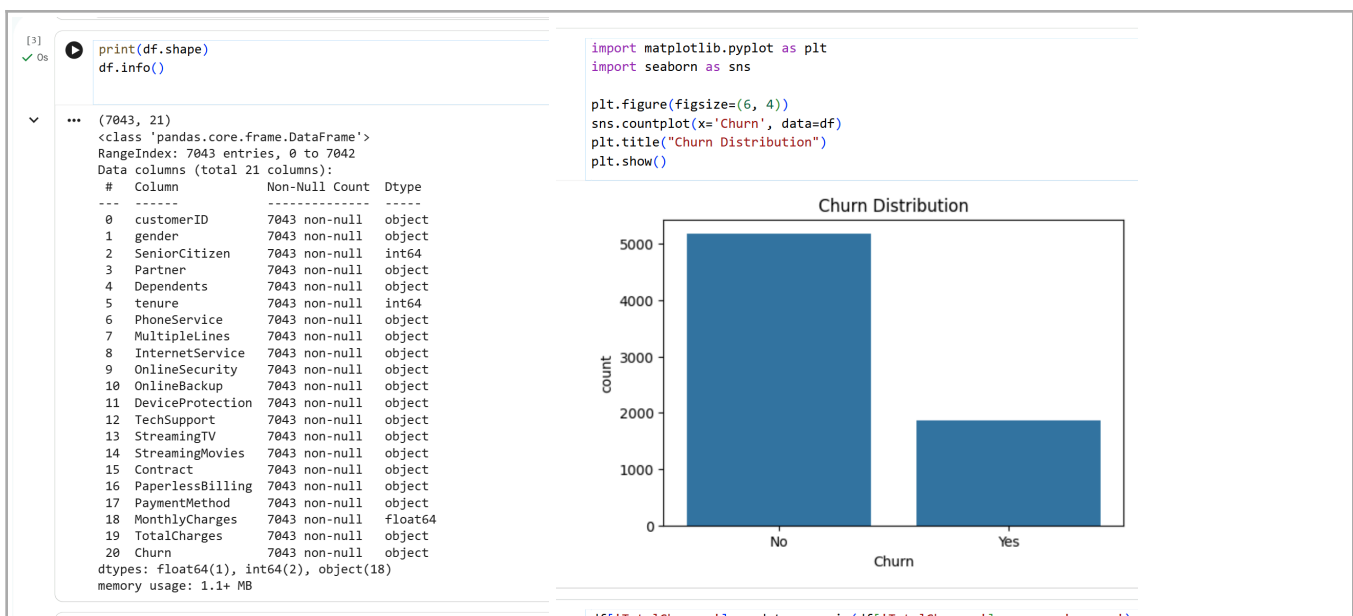
H.A.D.H. Kavindhya | S/21/084

1. Dataset

This project uses the Telco Customer Churn dataset, obtained from Kaggle:

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>. The dataset contains 7,043 customer records from a telecommunications company, with 21 attributes describing customer demographics (e.g. gender, senior citizen status), account information (e.g. tenure, contract type, payment method), subscribed services (e.g. internet service, streaming options), and monthly/total charges. The target variable, Churn, indicates whether the customer left the company (Yes/No).

The target class is imbalanced: 5,174 customers (73.5%) did not churn, while 1,869 customers (26.5%) churned. This imbalance motivated the use of precision, recall, and F1-score, rather than accuracy alone, to evaluate model performance, since a model that always predicts “no churn” would already achieve roughly 73% accuracy while providing no useful predictive value.



2. Preprocessing

- Converted the TotalCharges column from text to numeric; the 11 resulting missing values (new customers with zero tenure) were filled with 0.
- Dropped the customerID column, as it is a unique identifier with no predictive value.
- Encoded the target column Churn as a binary value (Yes → 1, No → 0).
- Applied one-hot encoding (pandas get_dummies) to all categorical feature columns (e.g. Contract, InternetService, PaymentMethod), expanding the feature set from 19 predictor columns to 45 numeric columns.
- Split the data into 80% training and 20% test sets using a stratified split, preserving the 73.5% / 26.5% churn ratio in both sets.
- Standardized all features using StandardScaler, fitted on the training set only and applied to both training and test sets, to avoid data leakage.

3. Models and Hyperparameter Optimization

Two classifiers were built and tuned as required: an XGBoost classifier and a Multilayer Perceptron (MLP). Both were trained on the same preprocessed, scaled training data and evaluated on the same held-out test set, allowing a direct comparison.

3.1 XGBoost

The XGBoost classifier was tuned using scikit-learn's GridSearchCV with 5-fold cross-validation, optimizing for F1-score. The search space covered $n_estimators \in \{100, 200\}$, $max_depth \in \{3, 5, 7\}$, and $learning_rate \in \{0.01, 0.1\}$, totalling 12 combinations (60 model fits with cross-validation).

3.2 Multilayer Perceptron (MLP)

The MLP was implemented in Keras as a feed-forward network with two hidden layers (ReLU activation) and a sigmoid output layer for binary classification, trained with the Adam optimizer and binary cross-entropy loss. Since Keras models do not integrate directly with GridSearchCV, hyperparameters were tuned via a manual search: four configurations varying hidden layer sizes ((32,16), (64,32), (128,64)) and learning rate (0.001, 0.01) were each trained on a validation split carved out of the training set, and the configuration with the highest validation F1-score was selected for final training.

3.3 Best Hyperparameters Found

Model	Best Hyperparameters Found
XGBoost (GridSearchCV, cv=5, scoring=F1)	learning_rate = 0.1 max_depth = 3 n_estimators = 100
MLP (manual search over 4 configs)	hidden layers = (32, 16) learning_rate = 0.001

The screenshot displays a Google Colab notebook titled "customer churn prediction.ipynb". The top section shows a code cell for XGBoost hyperparameter tuning using GridSearchCV. The code defines a parameter grid with `n_estimators` (100, 200), `max_depth` (3, 5, 7), and `learning_rate` (0.01, 0.1). It then creates an XGBoost classifier, performs a grid search with 5-fold cross-validation, and prints the best parameters and score. The output shows the best parameters are `learning_rate: 0.1`, `max_depth: 3`, and `n_estimators: 100`, with a best CV F1 score of 0.5895404644949563.

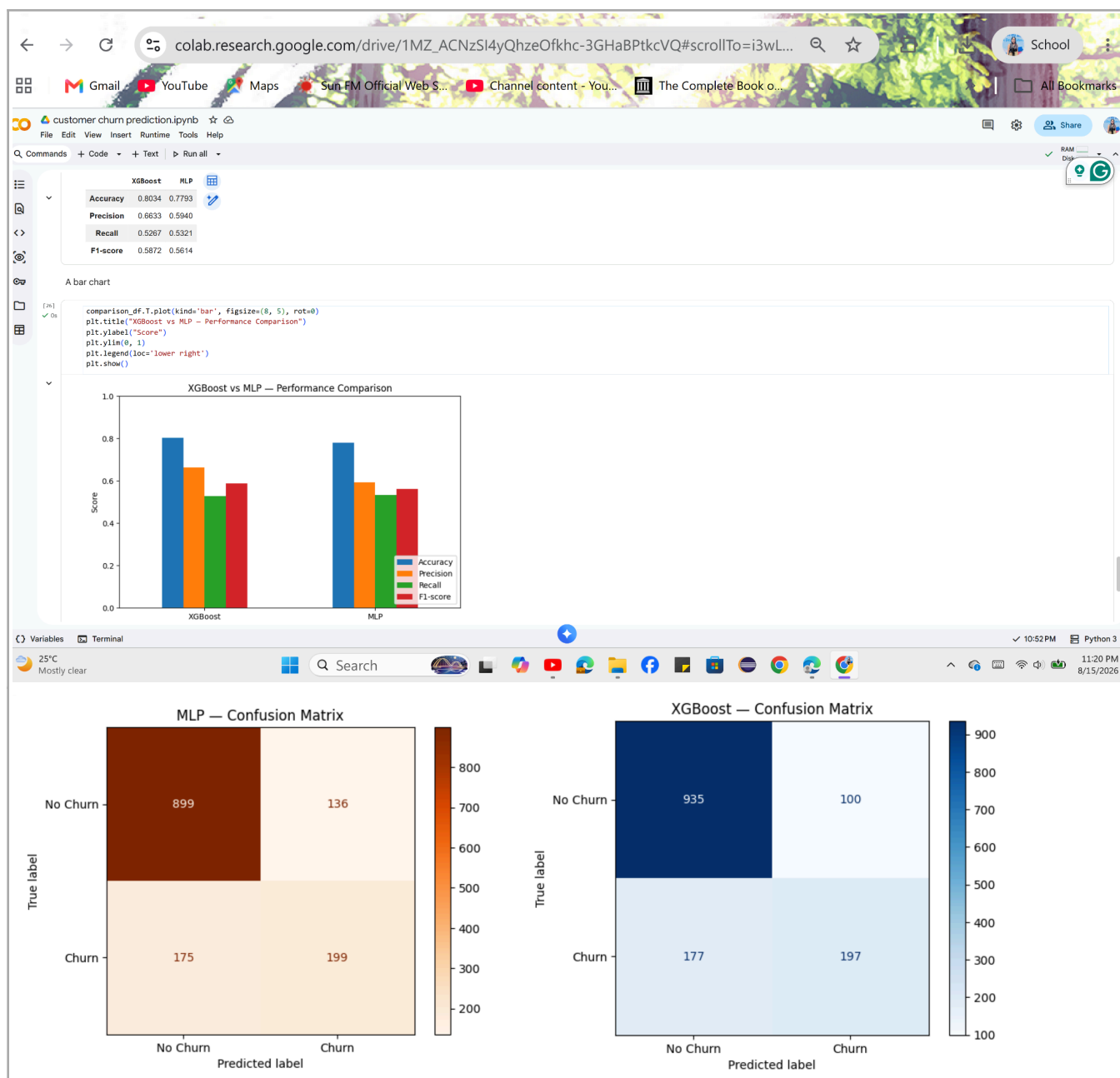
The bottom section shows another code cell for MLP hyperparameter tuning. It imports `train_test_split` and `f1_score` from sklearn. It carves a validation slice out of the training data and defines four configurations for hidden layer sizes and learning rates. A loop iterates over these configurations, builds an MLP model, fits it, and calculates the F1 score. The output shows the best configuration is `hidden1: 32, hidden2: 16, learning_rate: 0.001` with an F1 score of 0.5855379188712522.

The notebook interface includes a file explorer on the left, a command line, and a terminal at the bottom. The system tray at the very bottom shows the date and time as 8/15/2026, 11:15 PM.

4. Results

Both models were evaluated on the same held-out test set (1,409 customers). The table below summarizes accuracy, precision, recall, and F1-score for each model.

Metric	XGBoost	MLP
Accuracy	0.8034	0.7793
Precision	0.6633	0.5940
Recall	0.5267	0.5321
F1-score	0.5872	0.5614



5. Discussion

XGBoost outperformed the MLP on every metric except recall, where the two models were nearly identical (0.527 vs 0.532). XGBoost achieved a notably higher precision (0.663 vs 0.594), meaning that when it predicted a customer would churn, it was correct more often than the MLP. This translated into a higher overall F1-score for XGBoost (0.587 vs 0.561).

A likely explanation is that tree-based models such as XGBoost tend to perform well on tabular data with a mix of categorical and numeric features straight out of the box, since they can naturally handle non-linear feature interactions and are less sensitive to feature scaling. Neural networks such as the MLP typically require larger datasets and more extensive hyperparameter tuning to match tree-based methods on tabular classification tasks; given the relatively small dataset size (~7,000 rows) and the limited hyperparameter search performed here, it is plausible that a more exhaustive search or additional training epochs could narrow this gap.

Both models struggled more with recall than precision, meaning a meaningful share of customers who actually churned were not flagged. For a real business use case focused on proactive retention, this recall gap would be an important limitation to address — for example by adjusting the classification threshold or using class-weighting to penalize missed churners more heavily.

6. Conclusion

This project implemented and compared two machine learning pipelines for customer churn prediction on the Telco Customer Churn dataset: XGBoost and a Keras-based MLP, both with hyperparameter optimization. XGBoost achieved the stronger overall performance (F1 = 0.587 vs 0.561), likely owing to its suitability for structured tabular data. Both models nonetheless captured meaningful churn signals well above a naive baseline, and the comparison highlights the trade-offs between tree-based ensembles and neural networks on small-to-medium tabular datasets.

References

Kaggle. *Telco Customer Churn dataset*. <https://www.kaggle.com/datasets/blashtchar/telco-customer-churn>